

Evidencias de Uso

Q1: API de Biblioteca Virtual

Prova Tecnica — Dot Digital Group

Candidato: Silas Luiz Bom Fim

Data: 12/06/2026

Este documento apresenta evidencias de uso real da API de Biblioteca Virtual, desenvolvida com FastAPI + SQLAlchemy + SQLite. Cada secao mostra o formulario de requisicao e a resposta recebida da API, demonstrando o funcionamento correto de todos os endpoints.

1. Cadastrar Livro — POST /books/

Endpoint responsável por receber os dados de um novo livro e salvar no banco de dados. Retorna status 201 com o objeto criado, incluindo o ID gerado automaticamente.

POST /books/ Cadastrar livro

Cadastre um novo livro na biblioteca com título, autor, data de publicação e resumo.

Parameters Cancel Reset

No parameters

Request body required application/json

Edit Value | Schema

```
{  "title": "Dom Casmurro",  "author": "Machado de Assis",  "published_date": "1899-01-01",  "summary": "Um clássico da literatura brasileira."}
```

Execute Clear

Responses

Figura 1a — Formulário de cadastro com dados de Dom Casmurro (Machado de Assis)

Server response

Code	Details
201	Response body <pre>{ "title": "Dom Casmurro", "author": "Machado de Assis", "published_date": "1899-01-01", "summary": "Um clássico da literatura brasileira.", "id": 3}</pre> Response headers <pre>content-length: 140content-type: application/jsondate: Fri, 12 Jun 2026 03:18:16 GMTserver: uvicorn</pre>

Responses

Code	Description	Links
201	Successful Response	No links
	Media type application/json Controls Accept header. Example Value Schema <pre>{ "author": "J.R.R. Tolkien", "published_date": "1954-07-29", "summary": "Uma épica jornada pela Terra Média.", "title": "O Senhor dos Anéis"}</pre>	
422	Validation Error	No links

Figura 1b — Resposta 201 Created: livro cadastrado com sucesso, ID 3 gerado

2. Listar Todos os Livros — GET /books/

Endpoint que retorna a lista completa de livros cadastrados. Suporta filtros opcionais por título e autor. Sem filtros, retorna todos os registros com status 200.

GET /books/ Listar livros

Retorna todos os livros. Use ?title= ou ?author= para filtrar.

Parameters

Name

Description

title

Filtrar por título (busca parcial)

string | (string | null) (query)

author

Filtrar por autor (busca parcial)

string | (string | null) (query)

Execute

Clear

Responses

Figura 2a — Formulário GET /books/ sem filtros aplicados

Responses

Curl

```
curl -X 'GET' \
  'http://localhost:8080/books/' \
  -H 'accept: application/json'
```

Request URL

```
http://localhost:8080/books/
```

Server response

Code

Details

200

Response body

```
{
  {
    "title": "O Senhor dos Anéis",
    "author": "J.R.R. Tolkien",
    "published_date": "1954-07-29",
    "summary": "Uma épica jornada pela Terra Média.",
    "id": 1
  },
  {
    "title": "O Senhor dos Anéis",
    "author": "J.R.R. Tolkien",
    "published_date": "1954-07-29",
    "summary": "Uma épica jornada pela Terra Média.",
    "id": 2
  },
  {
    "title": "Dom Casmurro",
    "author": "Machado de Assis",
    "published_date": "1899-01-01",
    "summary": "Um clássico da literatura brasileira.",
    "id": 3
  }
}
```

Download

Response headers

```
content-length: 432
content-type: application/json
date: Fri, 12 Jun 2026 03:28:13 GMT
server: uvicorn
```

Figura 2b — Resposta 200 OK: lista com 3 livros cadastrados

3. Filtrar por Autor — GET /books/?author=Machado

O mesmo endpoint GET /books/ aceita o parametro ?author= para busca parcial sem diferenciar maiusculas/minusculas. Retorna apenas os livros cujo autor contenha o texto informado.

GET /books/ Listar livros

Retorna todos os livros. Use ?title= ou ?author= para filtrar.

Parameters

Name	Description
title string (string null) (query)	Filtrar por título (busca parcial) title
author string (string null) (query)	Filtrar por autor (busca parcial) Machado

Execute

Clear

Responses

Figura 3a — Formulário com filtro author=Machado

Responses

Curl

curl -X 'GET' \ 'http://localhost:8080/books/?author=Machado' \ -H 'accept: application/json'

Request URL

http://localhost:8080/books/?author=Machado

Server response

Code	Details
200	Response body <pre>{ { "title": "Dom Casmurro", "author": "Machado de Assis", "published_date": "1899-01-01", "summary": "Um clássico da literatura brasileira.", "id": 3 } }</pre> Download Response headers <pre>content-length: 142 content-type: application/json date: Fri, 12 Jun 2026 03:27:43 GMT server: uvicorn</pre>

Responses

Code	Description	Links
------	-------------	-------

Figura 3b — Resposta 200 OK: apenas Dom Casmurro retornado

4. Buscar Livro por ID — GET /books/1

Endpoint que retorna um livro específico pelo seu ID. Quando o ID existe no banco de dados, retorna os dados completos com status 200.

GET

/books/{book_id}

Buscar livro por ID

⌵

Retorna os dados de um livro específico pelo seu ID.

Parameters

Cancel

Name	Description
book_id required	
integer	
(path)	

Execute

Clear

Responses

Figura 4a — Formulário com book_id=1

Responses

Curl

```
curl -X 'GET' \
  'http://localhost:8000/books/1' \
  -H 'accept: application/json'
```

Request URL

http://localhost:8000/books/1

Server response

Code	Details
200	Response body <pre>{ "title": "O Senhor dos Anéis", "author": "J.R.R. Tolkien", "published_date": "1954-07-29", "summary": "Uma épica jornada pela Terra Média.", "id": 1 }</pre> Download Response headers <pre>content-length: 144 content-type: application/json date: Fri, 12 Jun 2026 03:28:41 GMT server: uvicorn</pre>

Responses

Figura 4b — Resposta 200 OK: O Senhor dos Aneis (Tolkien) retornado com sucesso

5. Livro Nao Encontrado — GET /books/5

Quando o ID informado nao existe no banco de dados, a API retorna status 404 com mensagem de erro em portugues, informando que o livro nao foi encontrado. Isso demonstra o tratamento de erros implementado no endpoint.

The screenshot shows a REST client interface with a GET request to `/books/{book_id}`. The description says "Buscar livro por ID" and "Retorna os dados de um livro especifico pelo seu ID." The parameters section shows a table with columns "Name" and "Description". There is one parameter: `book_id` with type `integer` and description `(path)`. The value `5` is entered in the input field. There are "Execute" and "Clear" buttons. The "Responses" section is empty.

Name	Description
<code>book_id</code> * required integer (path)	5

Figura 5a — Formulario com `book_id=5` (ID inexistente)

The screenshot shows the response of the GET request. The status is 404 Not Found. The response body is a JSON object: `{ "detail": "Livro com ID 5 não encontrado." }`. The response headers are: `content-length: 44`, `content-type: application/json`, `date: Fri, 12 Jun 2026 03:29:05 GMT`, and `server: uvicorn`.

```
curl -X 'GET' \
  'http://localhost:8000/books/5' \
  -H 'accept: application/json'
```

Request URL: `http://localhost:8000/books/5`

Server response

Code	Details
404 Undocumented	Error: Not Found

Response body

```
{
  "detail": "Livro com ID 5 não encontrado."
}
```

Response headers

```
content-length: 44
content-type: application/json
date: Fri, 12 Jun 2026 03:29:05 GMT
server: uvicorn
```

Figura 5b — Resposta 404 Not Found: "Livro com ID 5 nao encontrado."